

OMATrust Identity Registry Specification

Decentralized and Permissionless Trust Layer for the Open Internet

1. Executive Summary

This document (“Specification”) defines the requirements for the Identity Registry of OMATrust, a decentralized and permissionless trust layer for the open internet. It has two main components:

- Application Registry- a registry of tokenized Applications that are identified by DIDs.
- Ownership Resolver- a smart contract that resolves token ownership disputes.

This Specification will be referenced by and OMATrust Core Specification and works in conjunction with the OMATrust Proof Specification and OMATrust Reputation Specification.

The Specification is fully compliant with ERC-8004’s requirements for the Identity Registry. The specification details the on-chain and off-chain metadata for tokenized applications, the processes for metadata confirmation and ownership verification, and control policies for versioning.

2. Scope

2.1 In-Scope

This document aims to solve the following requirements and use cases described in the [Spatial Store RFP](#).

2.2 Out-of-Scope

Code repositories, tokenomics, business models, and storage implementations are out of scope for this document.

3. References

IWPS Identity Specification: <https://github.com/oma3dao/iwps-specification>

Spatial Store RFP: <https://github.com/oma3dao/spatial-store-rfp>

OMATrust Whitepaper: <https://github.com/oma3dao/omatrust-docs/blob/main/whitepaper.md>

DID Specification: <https://www.w3.org/TR/did-core>
DID Spec Registries: <https://www.w3.org/TR/did-spec-registries/>
Metaverse Standards Forum Spatial Store use case
Metaverse Standards Forum Autonomous Payments use case
OMATrust Proof Specification
ERC-8004 Specification

4. Definitions

4.1 Abbreviations

In the present document, the following abbreviations apply:

DID	Decentralized Identifier
DNS	Domain Name System
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
ID	Identifier
IdP	Identity Provider
RP	Relying Party
TTL	Time To Live
URL	Uniform Resource Locator
IWPS	Inter World Portaling System
VC	(W3C) Verifiable Credential

Relevant External Specification Terminology Sections

DID Spec	https://www.w3.org/TR/did-core/#terminology
VC Spec	https://www.w3.org/TR/vc-data-model-2.0/#terminology

The following applies to the specification:

- “string” means a UTF-8 string
- “URL” means a string in a specific format
- “JSON” means a string in JSON format
- [] is meant to signify an array of whatever is inside the brackets.

4.2 Definitions

The following definitions are used within the present document.

Term	Definition
Application	A software service that can come in many formats, including a self-contained software application on a Device, an API endpoint, or a smart contract.
Client	Software that queries OMATrust to obtain information on an Application.
Decentralized Identifier	An identifier that adheres to the W3C DID standard.
DID Document	A JSON-LD document containing information about the DID, such as public keys and services.
Owner	The entity that controls the address that owns the app token. Owner may or may not be the same entity that minted the token.
Issuer	An entity that issues credentials as attestations in the Reputation Service.

In addition to the above definitions all OMA3 specifications use requirements language as described in the [OMA3 working group process](#).

5. Specification

5.1 App Registry Contract

5.1.1 Onchain Metadata

The Application Registry contract tokenizes applications on the blockchain as an NFT. This document describes the specifications for this contract in a platform-agnostic manner.

Every app registry NFT stores the following information associated with an application.

Field	Format	Description	Req?	Mutable?
did	string	See Table 3	Y	N
fungibleTokenId	string	CAIP-19 token ID	N	N
contractId	string	CAIP-10 contract address	N	N
versionHistory	[object]	Array of released version structs, append only	Y	Y

status	enum	Active, deprecated, replaced	Y	Y
minter	address	Address of the transaction signer.	Y	N
owner	address	Current owner (built into ERC-721)	Y	Y (soulbound optional)
dataUrl	URL	URL to offchain data	Y	Y
dataHash	string	Hash of the JSON returned by dataUrl (see 5.1.3.3)	Y	Y
dataHashAlgorithm	string	The hash algorithm used to compute dataHash. Values: "keccak256", "sha256". Default is VM-specific (e.g.- keccak256 for EVM).	N	Y
traitHashes	[string]	A structure of hashed traits. Implementation is different for each VM. Implementations SHOULD cap on-chain traitHashes to ≤ 20 entries to mirror the off-chain keywords cap, and clients MUST NOT assume more than 20 are indexed. See Appendix C.	N	Y
interfaces	[enum]	An unordered set of interface capability codes. Multiple capabilities may be present. Example: if using a bitmap, bit 0 = human, bit 1 = api, and bit 2 = smart contract.	Y	N

Table 1: Application Registry Onchain Data.

All fields listed above require confirmation according to Section 5.1.3. Confirmation methods vary by field (e.g., ownership confirmation, attestation validation).

5.1.1.1 JSON Format: **versionHistory**

The objects in the **versionHistory** array have the following fields:

Value	Format	Required
major	Int	Y
minor	Int	Y
patch	Int	Y

5.1.1.2 JSON Format: **dataUrl**

dataUrl points to an endpoint that returns a JSON object with offchain data. The JSON object has the following top level fields depending on the value of the **interfaces** field in the NFT contract:

Value	Format	Description	Interface		
			0	2	4
name	string	App name. Matches ERC-721 metadata extension.	Y	Y	Y
external_url	URL	URL of the app's market website. Matches ERC-721 metadata extension.	O	O	O
image	URL	URL to the application icon. Matches ERC-721 metadata extension.	Y	O	O
description	string	Long description of the application. Max 4000 chars. Matches ERC-721 metadata extension.	Y	Y	Y
publisher	string	Publisher name.	Y	Y	Y
summary	string	Short description of the application. Max 80 chars.	O	O	O
owner	string	Address of the owner of the app token NFT. Used to confirm ownership of the dataUrl JSON (see "Field Confirmation" below). Format is CAIP-10.	Y	Y	Y
screenshotUrls	[URL]	JSON urls field contains an array of URLs to screenshot images.	Y	N	N
videoUrls	[URL]	JSON urls field contains an array of URLs to videos.	O	N	N

3dAssetUrls	[URL]	JSON urls field contains an array of URLs to 3D assets (GLB, USDZ, etc.).	O	N	N
legalUrl	URL	URL to legal agreements like license, terms of service, privacy policy, etc.	O	O	O
supportUrl	URL	URL to get support.	O	O	O
iwpsPortalUrl	URL	See IWPS Spec.	O	O	O
traits	[string]	An array of max 20 traits and the total char count cannot exceed 120. See Appendix C.	O	O	O
interfaceVersions	[string]	Array of supported versions of the interface	N	O	O
platforms	JSON	Object of platforms supported by the app.	Y	N	N
services	[JSON]	Array of services objects (see below)	N	Y	O
artifacts	JSON	Allows clients to verify content (e.g.- binaries)	O	O	N
registrations	[JSON]	Array of App Registry tokenizations	Y	Y	Y
version	string	x.y.z	O	O	O

Table 2: Application offchain data.

5.1.2 Offchain Metadata

5.1.2.1 JSON Format: **dataUrl.platforms**

If **interfaces** = 0, The **platforms** JSON object MUST contain one or more of the following fields depending on how the human user interacts with the app:

Value	Format	Description	Required
web	JSON	Web is supported	O
ios	JSON	iOS is supported	O
android	JSON	Android is supported	O
windows	JSON	Windows is supported	O
macos	JSON	MacOS is supported	O

meta	JSON	Meta Quest is supported	O
playstation	JSON	Playstation is supported	O
xbox	JSON	XBox is supported	O
nintendo	JSON	Nintendo is supported	O

A **platform** field is a JSON object that has the following fields:

Value	Format	Description	Required
launchUrl	string	URL to launch the app	Y
supported	[string]	iPhone, iPad, x64, arm64, etc.	O
downloadUrl	string	URL to download a binary	O
artifactDid	string	See Appendix A	O

5.1.2.2 JSON Format: **dataUrl.services**

The **services** field contains an array JSON objects, each of which contains the following fields:

Value	Format	Description	Required
name	string	MCP, A2A, etc.	Y
endpoint	string	URL of the endpoint	Y
version	string	Service-specific	O
schemaUrl	string	URL to API schema or documentation.	O

For **interface** = 4 (contracts), the chain ID is taken from the DID (did:pkh with CAIP-10 ID). Clients can then determine the format the RPC endpoint requires based on the chain ID.

There are additional option fields in **services** depending on the type of

5.1.2.3 Endpoint Type MCP

If an object in **dataUrl.services** has **name** equal to **MCP** (see 5.1.2.2) the endpoint object holds additional fields that represent the MCP specification and give agents the information they need to interface with an MCP server. The reader is referred to the specification of MCP v1.0 (modelcontextprotocol.io, Section 3, Server Metadata), which has descriptions of each field and is incorporated by reference.

The following are the additional JSON fields for an **services** object:

Value	Format	Description	Required
tools	[JSON]	See below	Y
resources	[JSON]	See below	Y
prompts	[JSON]	See below	Y
transport	JSON	See below	Y
authentication	JSON	See below	Y

5.1.2.3.1 MCP JSON Format: **dataUrl.services[0].tools**

The **tools** array contains JSON objects with the following fields:

Value	Format	Description	Required
name	string		Y
description	string		Y
inputSchema	JSON		Y
annotations	JSON		N

5.1.2.3.2 MCP JSON Format: **dataUrl.services[0].resources**

The **resources** array contains JSON objects with the following fields:

Value	Format	Description	Required
-------	--------	-------------	----------

uri	string		Y
name	string		Y
description	string		N
contentType	string		N

5.1.2.3.3 MCP JSON Format: **dataUrl.services[0].prompts**

The **prompts** array contains JSON objects with the following fields:

Value	Format	Description	Required
name	string		Y
description	string		Y
arguments	[JSON]		N

5.1.2.3.4 MCP JSON Format: **dataUrl.services[0].transport**

The **transport** JSON object contains the following fields:

Value	Format	Description	Required
http	JSON		N
stdio	JSON		N

5.1.2.3.5 MCP JSON Format: **dataUrl.services[0].authentication**

The **authentication** JSON object contains the following fields:

Value	Format	Description	Required
oauth2	JSON		N

blockchain	JSON		N
------------	------	--	---

5.1.2.4 JSON Format: **dataUrl.artifacts**

artifacts is a JSON map keyed by the value of **artifactDid** (see Appendix A). It carries integrity and distribution details for any verifiable payload referenced from **dataUrl.platforms**.

Key format: **did:artifact:<cidv1>** (CIDv1 base32 of the artifact bytes)

The most prevalent use is for downloadable binaries as it allows clients to check the legitimacy of a binary before downloading it. This object reflects supply-chain lessons baked into Apple notarisation, Windows Authenticode, Sigstore, and SLSA.

The following table shows the common fields in each object in the mapping.

Value	Format	Description	Required
type	string	"binary", "container", or "website"	Y
sriManifest	JSON	See Appendix B	N
os	string	"windows", "macos", "linux"	Y (except website)
architecture	string	"x64", "arm64"	Y (except website)
contentType	string		N
downloadUrls	[string]	Array of download URLs	N
sizeBytes	Integer		N
provenanceUrl	URL	SLSA provenance JSON	N
signatureUrls	[JSON]	Example: [{"sigUrl": "blah", "sigAlgorithm": "pgp/x509"}]	N

otherDigests	[JSON]	Example: [{"hash": "blah", "algo": "sha256"}]	N
notarization	TBD		N
transparencyLog	JSON	See below	N

JSON example for the **artifacts** field:

JSON

```
{
  "did:artifact:<cidv1>": {
    "type": "binary",
    "os": "windows",
    "architecture": "x64",
    "mimeType": "application/vnd.microsoft.installer",
    "downloadUrls": [
      "https://dl.oma3.dev/pkg/app-v2-windows.exe"
    ],
    "sizeBytes": 1582034,
    "provenanceUrl": "https://dl.oma3.dev/pkg/app-v2-slsa.json",
    "signatureUrls": [
      {
        "sigUrl": "https://dl.oma3.dev/pkg/app-v2-windows.exe.sig",
        "sigAlgorithm": "x509",
        "keyLocator": "did:web:example.com#release-key"
      }
    ],
    "otherDigests": [
      {
        "hash": "e05ac4bb...",
        "algo": "blake3",
        "canon": "raw"
      },
      {
        "hash": "8b7e1a...",
        "algo": "sha512"
      }
    ]
  }
}
```

```

    }
  ],
  "transparencyLog": {
    "type": "rekor",
    "entryId": "sha256:bf33..."
  }
}
}

```

5.1.2.5 JSON Format: **dataUrl.registrations**

registrations is a JSON array of objects, as defined in the ERC-8004 specification, that has the following fields:

Value	Format	Description	Required
agentId	integer	Must equal the NFT tokenId	Y
agentRegistry	string	CAIP-10 ID of the NFT contract	Y

5.1.3 Metadata Confirmation

OMATrust uses several mechanisms to confirm the validity of data stored in app tokens. Some of these mechanisms are performed by the registry itself, and other mechanisms are performed by the client.

5.1.3.1 **did** Confirmation

OMATrust requires that the developer minting the NFT controls the **did**. Although this requires more effort from the developer, it results in higher trust in the system. **did** ownership confirmation consists of the following steps:

1. The app owner proves to an approved Issuer that it owns the **did**. See below for proof mechanisms based on the format of the ID.
2. Issuer submits an attestation to the Resolver contract (Section 5.2).

3. Registry contract calls the Resolver contract. If the proper attestations are in place, the token is registered. If not, the registration fails. See Section 5.1.6 for how conflicts, TTL, and revocations are handled.

Clients MAY also check ownership of **did** and other metadata values. The following sections describe how ownership is proven for various types of IDs.

Note: Ownership confirmation occurs at mint time. However, control of a contract or token can change later (for example, a proxy admin may be transferred, or a mint authority may be revoked). Clients that depend on ongoing correctness SHOULD re-verify ownership through the Resolver (see Section 5.1.6) if these changes are relevant to their use case.

5.1.3.1.1 **did:web** Confirmation

If the format of **did** is **did:web**, the owner MUST support one of the following methods to verify their ownership of the domain.

- **did:web URL**: owners MUST return a JSON object at the following URL, where url is the URL specified in the **did**: **[url]/.well-known/did.json**.

This object is called a DID Document. Here is an example DID Document:

JSON

```
{
  "@context": [
    "https://www.w3.org/ns/did/v1",
    "https://w3id.org/security/suites/secp256k1recovery-2020/v2"
  ],
  "id": "did:web:example.com",
  "verificationMethod": [
    {
      "id": "did:web:example.com#app-owner-key",
      "type": "EcdsaSecp256k1RecoveryMethod2020",
      "controller": "did:web:example.com",
      "blockchainAccountId":
        "eip155:1:0x89a932207c485f85226d86f7cd486a89a24fcc12"
    }
  ],
  "authentication": [
    "did:web:example.com#app-owner-key"
```

```
]
}
```

To verify this method, the Issuer MUST retrieve the DID Document located at **[url]/.well-known/did.json** and verify that the owner address appears as an array element of the **verificationMethod** field in the DID Document returned by the endpoint. Note that for purposes of **did:web** ownership verification, OMATrust first normalizes the host component of the DID by lowercasing it, removing any trailing **.**, and stripping a leading **www**. For example, **did:web:www.example.com** and **did:web:example.com** are treated as the same canonical subject.

- **dns:<domain>**: The identifier "dns:<domain>" asserts control of <domain> via a DNS TXT record. The Owner MUST publish a TXT record at: **_controllers.<domain>**. The TXT value MUST be a sequence of key=value pairs separated by semicolons or spaces:

v=1 (protocol version, fixed to "1")
controller= (one or more identifiers in DID format)

Examples:

None

```
v=1;controller=did:pkh:eip155:1:0x89a932207c485f85226d86f7cd...
v=1 controller=did:pkh:eip155:1:0x11...
controller=did:pkh:eip155:1:0x22
```

Multiple **controller** values indicate co-controllers. For rotation, both old and new controllers SHOULD be published during an overlap period. If the apex cannot be modified, the record MAY be published on a subdomain (e.g., **id.example.com**), in which case the identifier string is **did:web:id.example.com**. Issuers MUST query authoritative name servers and SHOULD validate DNSSEC when available. The minting wallet MUST match one of the **controller** values to prove ownership.

5.1.3.1.2 **did:pkh** Confirmation

This DID method is used to tokenize a smart contract application. Smart contracts do not need to be tokenized in order for users to file attestations on them. Tokenizing a smart contract is primarily for discovery and usage information.

If a smart contract is tokenized, the Issuer MUST confirm that the address minting the smart contract token is controlled by the same entity that either administered or deployed the contract. For EVM contracts:

- If the contract has an admin, the admin control address MUST be used.
- If the contract is immutable, the deploying address MUST be used.

There are several ways verification can be done:

5.1.3.1.2.1 Automated Address Matching

1. Determine the controlling address of the contract
 - Upgradeable contracts:
 - Inspect the contract's metadata or storage to locate the address that holds upgrade authority.
 - If an intermediate controller contract exists (e.g., an admin module), resolve the controlling address from that contract.
 - Contracts with explicit owner/admin roles:
 - Query the contract for the address associated with its owner or administrative role.
 - Immutable contracts:
 - If no admin or owner role is detectable, the deploying account from the creation transaction is the controlling address.
(Example: On Ethereum, this may be read from well-known storage slots such as **eip1967.proxy.admin**, or via calls to functions like **owner()** or role checks in **AccessControl**.)
2. Compare with the minter of the registry token
 - The address that minted the registry token MUST equal the controlling address derived above.
 - If they match, ownership is verified.

5.1.3.1.2.2 Onchain Transfers

This method allows a controlling wallet to grant delegate access to another (e.g.- minting) wallet by sending a deterministic, minimal native-asset transfer. The transfer constitutes an OMATrust Proof of type **tx-encoded-value** as specified in the OMATrust Proof Specification.

Issuers and verifiers MUST use the OMATrust Proof Specification **proofType = tx-encoded-value** (OMATrust Proof Specification §5.3.6) with **proofPurpose =**

shared-control (OMATrust Proof Specification §5.1.3.1) to implement and validate this method. All rules for Proof construction, valid transaction form, chain selection, and verification are defined in the Proof Specification and are not repeated here.

The following can be used as a guide when implementing **tx-encoded-value**:

- Subject/sender: the onchain object delegating access (**did**, **contractId**, etc.)
- Controller/recipient: the address receiving the delegation (**minter**, **owner**, etc.).

Notes:

- “Controller” in the context of a Proof is different from the controlling address of a Subject. Controller is the entity receiving the delegate access. The controlling address of a Subject could be found using the mechanisms described in 5.1.3.1.2.
- This method creates a public, verifiable onchain association between the Subject and Controller. Developers should treat this link as public proof of control and avoid using high-security treasury addresses for this purpose when possible.

5.1.3.1.2.3 Manual Confirmation

If a smart contract is not EVM-compatible or if the registry token is to be minted by a different address (e.g., a multisig, DAO, or delegated key) than the controlling address, the Issuer **MUST** use another mechanism to discern if the entity that owns the minting address also controls the controlling address. Manual mechanisms include:

- Asking the entity to sign a transaction with both wallets that includes a randomly generated nonce, such as making a payment in the amount of the nonce.
- Doing a thorough background check to see if the entity's representative is an approved member of the organization that controls the controlling address.

Once the proof has been given, the issuer **MUST** make an attestation binding the minting address to the controlling address (Section 5.3.5.1). Clients **SHOULD** accept such an attestation only if it is issued by a trusted Issuer using an accepted verification mechanism. If no match is found and no valid attestation is present, clients **MUST** treat the tokenized contract as unverified.

5.1.3.2 **contractId** and **fungibleTokenId** Confirmation

These CAIP-10 fields **SHOULD** be confirmed by the client using the same mechanisms the Issuer follows when confirming a did:pkh DID as described in Section 5.1.3.1.2.

5.1.3.3 **dataURL** Confirmation

5.1.3.3.1 **dataHash** Check

Clients MUST fetch **dataUrl**, canonicalize the returned JSON (see 5.1.3.4.2), compute the digest of the the resulting UTF-8 bytes using the on-chain **dataHashAlgorithm** (see 5.1.3.4.3), and compare it to the on-chain **dataHash** value. If they differ, clients MUST treat the manifest as unverified and SHOULD display a warning or hide the app according to UI policy

5.1.3.3.2 **dataURL** Ownership

The JSON returned by the **dataUrl** API endpoint MUST contain the **owner** field, and its value MUST match the NFT owner address. Furthermore, the owner SHOULD ensure the existence of a trusted third-party attestation that verifies certain dataUrl fields such as:

- name
- publisher
- summary
- description
- image
- screenshotUrls
- videoUrls
- 3dAssetUrls
- external_url

5.1.3.3.3 Other URL Confirmation

The dataUrl object could contain URL fields. The client MAY confirm the ownership of these URLs as well using one of the following mechanisms:

- URLs that use a domain that has already been verified in Section 5.1.3.1.1 or Section 5.1.3.3.2 do not need to be re-verified.
- All other URLs MAY be verified using the same mechanism to verify did:web DIDs or by checking a Linked Identifier attestation (Section 5.3.5.1).

For URLs that point to media:

- Clients SHOULD place higher trust in content addressable URLs such as IPFS or Filecoin URLs, as the content in these URLs cannot be changed without changing the URL.
- If the URL returns a media file, the media file MAY have the owner address embedded in the file in some manner.

5.1.3.4 JSON Policies

5.1.3.4.1 JSON Parsing

Parsing JSON objects MUST conform to RFC 8259 (ECMA-404). Inputs that contain comments, single quotes, NaN/Infinity, trailing commas, or other non-standard extensions MUST be rejected.

5.1.3.4.2 JSON Canonicalization

To ensure deterministic **dataHash** values across implementations, the **dataUrl** MUST be canonicalized using JCS (RFC 8785). Canonicalization (JCS, RFC 8785) summary:

- Object member names are sorted lexicographically (by Unicode code point).
- Objects are emitted in sorted order; array element order is preserved.
- Numbers are emitted in their minimal form (no leading zeros, no superfluous decimal points or exponents; -0 normalizes to 0).
- Strings are escaped exactly as specified in RFC 8785; no additional Unicode normalization is applied.
- All insignificant whitespace is removed.

5.1.3.4.3 JSON Hashing

dataHash = **HASH(canonicalUtf8Bytes)** where **HASH** is the algorithm specified in **dataHashAlgorithm**. The algorithm used MUST be recorded in **dataHashAlgorithm** and the digest MUST be encoded as a 0x-prefixed lowercase hex string.

5.1.3.4.3 Other Guidance

- Prefer integers and strings over floats where precision matters to avoid cross-runtime number formatting issues.
- Implementers SHOULD publish golden test vectors for canonicalization+hashing (covering numbers, escape sequences, object key ordering, and nested structures) and verify cross-runtime determinism to ensure consistency across implementations. Clients are encouraged to use these vectors to validate their consumption of OMATrust data. See Appendix D for example test vectors to guide implementation.

Note: Contracts do not validate JCS; they only store the algorithm + digest. JCS compliance and hash correctness are enforced off-chain by clients and indexers.

5.1.4 Control Policy

App Registry contracts **MUST** manage when version updates are required and when new NFT mints are required. These policies are based on policies of existing app stores that have years of experience mitigating fraudulent behavior.

- Each DID requires a different token in the Registry. It is the equivalent of Apple's Bundle ID.
- Different major versions of a specific DID also require a new token.

The following table details versioning rules for certain onchain fields.

Desired Change	On-chain rule
Move from (did, major) → (did, major+i)	Must mint a new NFT
Edit interfaces	Interfaces change requires minor+i and must be additive only
Edit traitHashes	Requires patch+i or minor+i
Edit dataUrl or fungibleTokenId	Must mint a new DID
Edit contractId	Not allowed
Transfer NFT ownership	Allowed without version changes

Table 3: Allowed DID methods for Application Registry DID field.

Control Policy Justification

- Apple's App Store ties immutability to the Bundle ID, forbidding any Bundle ID change once an app is live, so users always know they're running the same canonical app. Google Play enforces the same rule on the Android Package Name, requiring publishers to create an entirely new listing if they change it.
- Swapping the fungibleTokenId on-chain is equivalent to issuing an infinite-mint ERC-20 rug pull, so we freeze that field and demand a new DID/NFT to alter it.

- Because CAIP-19 encodes the contract address in the asset ID, any breaking executable change must mint a new NFT keyed by (did,major) to maintain deterministic asset lookups.
- Non-breaking API additions only require a minor bump in version number, signaling backward compatibility, while metadata or binary tweaks controlled by dataHash require only a patch bump.

5.1.5 Optional Soulbound Mode

When enabled, transfers and approvals **MUST** be rejected; minting and burning remain allowed.

API impact: No new fields are required (mode can be implicit or a boolean per token), but behavior **MUST** be observable via a read method, e.g., `isSoulbound(tokenId)`.

UI guidance: Stores **SHOULD** visually label soulbound apps.

5.2 Ownership Resolver Contract

DID ownership is verified at token minting time. OMA3 uses a dedicated Resolver contract to confirm ownership and arbitrate conflicts when multiple parties claim to own the same DID.

Process

1. **First Attestation:** The owner of a DID gets an attestation from an approved Issuer to that confirms ownership of the DID. This confirmation process can be manual (e.g.- in conjunction with a cybersecurity audit) or automatic (e.g.- a server that checks `.well-known/did.json` programmatically). The Resolver contract holds the attestation onchain.
2. **First Mint:** With the ownership attestation in place, the owner mints the application with the wallet address in the attestation. The Resolver contract checks the attestation before confirming the mint.
3. **Challenge:** A challenger may attempt to rebind a DID by minting the same DID/version combination.
 - The challenger **MUST** either ask the original Issuer to reverse the attestation (issuing a new attestation) or enlist at least two other approved issuers to attest to the challenger's ownership of the DID in question. The Resolver compares the challenger's attestation score against the incumbent's.

- Scores are based on the count of valid, non-revoked attestations from an approved list of issuers.
- Only attestations older than a global maturation delay (e.g., 72h) count toward scores, creating a rolling challenge window.

4. **Resolution:** If the number of challenger attestations strictly exceed incumbent's, ownership flips. Otherwise, the challenge fails.

Attestation model

- Attestations are EIP-712 signed by approved attesters (starting with the OMA3 verification server).
- Each (**attester**, **did**, **epoch**, **claimer**) has only one active attestation, which can be updated or revoked.
- Events record every **AttestationUpdated**, **ConflictAttempted**, and **OwnershipChanged**.

Notification

- Incumbents receive notice via events; off-chain watchers (or push notification services) subscribe to these logs.
- An optional notifier callback can be registered by incumbents for on-chain hooks. This model ensures predictable resolution without upgradeable registries, while allowing flexible evolution of conflict policy.

5.3 Reputation Service

A permissionless app registry without third party attestations leaves users open to fraud. Adversaries can register malicious, fraudulent, misleading, and counterfeit apps. To address this problem OMATrust leverages the Reputation System.

5.3.1 OMATrust Reputation System

The OMATrust Reputation System leverages and augments existing services like Ethereum Attestation Service. It is comprised of the following components:

1. **Attestation Schemas:** OMA3 defines several schemas for different attestations, from user reviews to cybersecurity certifications. These are defined in the OMATrust Reputation Specification (§6–7).
2. **Cross Chain Addresses:** Instead of using chain-specific blockchain addresses to identify the attestation subject, OMATrust uses hashed DIDs, which support web domains as well as blockchain addresses.

3. Resolver: the Resolver contract stores onchain attestations related to ownership, as described above.

5.3.2 DID → DID Address Mapping and Searching (EVM)

OMATrust leverages Ethereum Attestation Service (EAS) on EVM chains. EAS identifies the subject of an attestation using an address-typed recipient field. Because DIDs are variable-length strings, OMATrust defines a deterministic mapping from a DID to an Ethereum address, called a DID Address, solely for use with EAS.

This address is an indexing label only. It is not an identity, does not imply control or ownership, and MUST NOT be interpreted as a wallet or transaction destination.

Computing the DID Address:

For any **did**, the corresponding EAS recipient address is derived as follows:

1. Canonicalize the DID:
 - a. Apply normalization rules defined by the DID method.
 - b. Examples:
 - i. **did:web**: lowercase the host, strip any leading **www** and trailing **.**, apply IDNA/punycode for international domains, preserve the path.
 - ii. **did:pkh**: use the canonical CAIP-10 chain/account encoding.
2. Compute the **didHash**
 - a. The **didHash** is the keccak256 digest of the canonicalized DID string
 - b. This produces a bytes32-equivalent hex string
3. Truncate to an Ethereum address
 - a. The DID Address is the low-order 160 bits of **didHash**

Output Encoding:

- The DID Address is defined conceptually as a 20-byte value (bytes20).
- When serialized as a string (e.g., in SDKs, APIs, logs, and test vectors) the DID Address MUST be encoded as:
 - a ``0x``-prefixed lowercase hexadecimal string
 - exactly 40 hexadecimal characters (20 bytes)
- EIP-55 checksum casing MUST NOT be required for interoperability.

5.3.3 Attestation Querying

Clients can retrieve attestations related to a DID by computing its DID Address and filtering attestations accordingly.

Example query flow:

1. Compute **indexAddress(did)** using the algorithm above.
2. Query EAS for attestations with **recipient = indexAddress(did)** and a given schema UID (e.g., **Oma3UserReview@1**).
3. Within each attestation payload, confirm that **subjectDidHash** matches the DID hash used to derive the recipient. This prevents mismatches or spoofing.

5.3.4 EAS Integration: Recipient Rule

When storing an attestation about a DID in EAS:

- The **recipient** field MUST equal the computed **indexAddress(did)**.
- The attestation payload MUST include **subjectDidHash**, which is exactly the **didHash** derived during DID Address computation (see Section 5.1.3.4.3)

Example Schema: **Oma3UserReview@1**

A review attestation payload might be structured as:

None

```
struct UserReviewPayload {
    uint8 rating; // Rating: 0-100 (basis points) or scaled 0-5 × 20
    bytes32 contentHash; // Hash or CID digest of the review content
    bytes5 locale; // Optional: BCP-47 locale code (e.g., "en-US")
    uint16 version; // Optional: schema/content version indicator
    bytes32 subjectDidHash; // MUST equal the didHash used in recip...
}
```

Discovery Pattern

To fetch reviews about a DID:

None

```
address recipient = indexAddress(did);
// Query EAS for attestations where:
// schemaUID == Oma3UserReview@1
// recipient == indexAddress(did)
```

Then decode the attestation payloads and resolve **contentHash** for off-chain content when needed.

Clients can retrieve attestations related to a DID by computing its DID Address and filtering attestations accordingly.

5.3.5 Reputation Specification (External Reference)

The OMATrust specification defines the identity, integrity, and attestation-plumbing components used across the ecosystem. These include DID resolution, data integrity verification, canonicalization rules, attestation indexing, and the integration requirements for onchain attestation systems.

Application-layer reputation semantics—such as user reviews, service-proof objects, endorsements, certifications, linked identifiers, and other trust-related attestations—are standardized separately in the OMA3 Reputation Specification.

The OMA3 Reputation Specification defines:

- the structure and semantics of all reputation-related attestations
- schemas for user reviews, endorsements, certifications, security assessments, and related reputation attestations; and rules for attestation-layer proof usage
- verification and trust-model requirements for attesters and relying parties
- client-side guidance for validation, aggregation, scoring, and display
- indexing and search conventions tailored for reputation data

Separating reputation semantics from the OMATrust core ensures that the identity and integrity layer remains stable and slow-moving, while the reputation layer can evolve independently and incorporate new attestation types, protocol extensions, and ecosystem practices over time.

Specifications on the below schemas can be found in the [OMA3 Reputation Service draft proposal](#).

5.3.6 Client Guidance

To ensure integrity and prevent user harm from fraudulent or misleading app metadata, all clients consuming data from OMATrust SHOULD adopt an attestation-based trust policy. An app store SHOULD NOT display any application token unless the following conditions are met:

1. **dataUrl** resolves to valid JSON conforming to the current offchain schema.

2. The SHA-256 (or Keccak-256) hash of the fetched JSON exactly matches the on-chain **dataHash** for the token.
3. A valid third-party attestation has been published referencing that specific **dataHash** and using an accepted schema(s).

Each app store may define its own attestation trust model. For example:

Model	Description
Single attester	A centralized store operator acts as sole reviewer (like Apple).
Multi-attester	Accept attestations from any approved third party (e.g., Mozilla, OMA3, community DAOs).
AI + human fallback	Require attestation from an automated reviewer unless the confidence score is low, in which case fallback to a human attester.
Community attestations	Support attestations from public reviewers with onchain reputation systems.

If an application does not have a valid attestation:

- It SHOULD be hidden by default from public listings.
- It MAY be shown with a warning or behind a user-controlled setting (e.g., “Show unverified apps” toggle).
- It SHOULD NOT be eligible for ranking, recommendation, or featuring.

This approach balances decentralized publishing with user trust, enabling permissionless participation while minimizing abuse.

Change History

Version	Date	Comments
0.1	2025-09-25	Initial draft - Alfred Tom
0.2	2025-09-28	Clarified data confirmation mechanisms
0.3	2025-10-02	Introduced TXT DNS domain verification

0.4	2025-10-11	Removed 5.1.6, dataUrl.endpoint.format, dataUrl.a2a, and payments
0.5	2025-10-30	Appendix C, _omatrust clarification, more did:pkh ownership confirmation methods, ERC-8004 compatibility.
0.6	2025-12-10	Added dataUrl.registrations and version, clarified default hash algorithms and that summary is optional, refinements to artifacts, require new NFT mint on dataUrl change.
0.7	2026-01-10	Simplified computation of DID Index Address and renamed to DID Address.
0.71	2026-02-10	Add did:handle specification appendix.
0.72	2026-02-15	Clarified DID Address format.
0.73	2026-04-01	Changed _omatrust to _controllers.
0.8	2026-04-08	Support latest version of ERC-8004 Specification.
0.81	2026-04-10	Limit DID Address to EVM-specific sections.
0.82	2026-04-23	Change definition of DID Address to strip www subdomains.
0.83	2026-06-20	Move did:artifact into separate spec.
0.84	2026-06-23	Complete Appendix D with hashes.

Appendix A

did:artifact Use

A.1 Integration Hooks

- **dataUrl.platforms:**
 - **launchUrl** (required) is the user-facing deep link/store page.
 - **artifactDid** (optional) SHOULD be present when a platform ships verifiable bytes (desktop installer/CLI/container) or a website proof artifact.
 - If **artifactDid** is present, a matching entry MUST exist at **dataUrl.artifacts[artifactDid]**.
- **dataUrl.artifacts:** A map keyed by **did:artifact:<cidv1>** per the standalone specification.

A.2 DID Address Helper

Reference Solidity helper for computing the DID Address from a **didHash**:

None

```
library DidIndex {
    function toAddress(bytes32 didHash) internal pure returns
(address) {
    return address(uint160(uint256(didHash)));
    }
}
```

Note: The returned address is an index label, not a controller. Never infer control or send assets to it.

A.3 Examples

Binary (Windows installer)

JSON

```
"platforms": {
  "windows": {
```

```

    "launchUrl": "myapp://open",
    "preferredDownloadUrl": "https://cdn.example.com/AppSetup.exe",
    "artifactDid": "did:artifact:<cidv1>"
  }
},
"artifacts": {
  "did:artifact:<cidv1>": {
    "type": "binary",
    "os": "windows", "arch": "x64",
    "sizeBytes": 73400320,
    "distributionURIs": [
      "https://cdn.example.com/AppSetup.exe",
      "ipfs://<cidv1>"
    ],
    "signatureURIs": ["https://sig.example.com/win.sig"],
    "algo": "sha256",
    "digestHex": "<sha256-hex>"
  }
}

```

None

did:artifact:<cidv1>

- **<cidv1>** MUST be CIDv1 encoded with multibase base32-lower.
- The CID's multihash MUST use SHA-256 under this spec version (5.1.3.4.3).
- The multicodec SHOULD be **raw** for opaque bytes. Using a more specific codec does not change verification semantics.

Website (SRI manifest)

JSON

```

"platforms": {
  "web": {
    "launchUrl": "https://play.example.com",
    "artifactDid": "did:artifact:<cidv1-of-jcs-sri-manifest>"
  }
}

```

```
    }  
  },  
  "artifacts": {  
    "did:artifact:<cidv1-of-jcs-sri-manifest>": {  
      "type": "website",  
      "originDid": "did:web:example.com",  
      "sriManifest": {  
        "version": 1,  
        "origin": "example.com",  
        "algo": "sha512",  
        "hashEncoding": "base64",  
        "paths": {  
          "/assets/app.css": "sha512-...",  
          "/assets/app.js": "sha512-..."  
        }  
      }  
    }  
  }  
}
```

Appendix B

Website Artifacts and SRI Manifests Draft Notes

This appendix captures design intent for website verification to be finalized in future version of the specification. It is for informational purposes only.

B.1 Summary (design intent)

- Website proofs will use **artifactDid** = **did:artifact:<cidv1>** (same as binaries).
- The website proof artifact will be an SRI manifest (JSON) whose JCS-canonicalized bytes are hashed (SHA-256 → CIDv1) to form the **artifactDid**.
- **platforms.web.launchUrl** remains the UX entry point;
platforms.web.artifactDid (optional for v1) can point to the website proof artifact when available.

B.2 Planned artifact shape (non-normative)

JSON

```
"artifacts": {
  "did:artifact:<cidv1-of-sri-manifest>": {
    "type": "website",
    "sriManifest": {
      "version": 1,
      "origin": "example.com",           // apex domain
      "algo": "sha512",
      "hashEncoding": "base64",
      "paths": { "/assets/app.js": "sha512-..." }
    },
    "downloadUrls": ["ipfs://<cidv1-of-sri-manifest>"] // optional
  }
}
```

Notes:

- The **origin** inside the manifest binds it to the domain and is part of the hashed bytes.

- No separate **originDid** is required in v1 draft; the client already knows the domain via **platforms.web.launchUrl**.

B.3 Planned verification algorithm

1. Resolve **artifactDid** from **platforms.web.artifactDid**.
2. Obtain manifest bytes (inline **sriManifest** or fetch via **downloadUrls**).
3. **JCS** canonicalize → **SHA-256** → **CIDv1**; require equality with **artifactDid**.
4. Ensure **registrableDomain(platforms.web.launchUrl) == sriManifest.origin**.
5. When rendering the site, validate loaded assets against manifest SRI entries.

B.4 Open items (track in issues)

- Deterministic packaging & test vectors for SRI manifests (JCS inputs/outputs).
- Handling multi-origin assets (likely out of scope for v1 web verification).
- Optional “snapshot” artifact (tar/warc) and how to embed origin metadata inside the archive if added later.
- UX guidance for partial verification (some assets verified, others not).
- Attestation linkage: publisher → domain (**ProofOfWebControl**) vs. publisher → artifact (**ArtifactBinding**).

Appendix C

Recommended Traits and traitHashes

traitHashes are optionally stored onchain to enable filtering of apps for certain onchain clients such as onchain autonomous agents. This appendix provides a list of recommended trait strings for use in the on-chain **traitHashes** array and the **dataUrl.traits** field. For the onchain field, developers must hash these values with the canonical hashing algorithm of the chain. For Ethereum this is keccak-256. Additional recommended traits may be proposed via OMA3 governance processes. Developers can add any string they want.

Trait String	Description
"api:openapi"	Include this if the interface field has a value of 2 and the API format is OpenAPI.
"api:graphql"	Include this if the interface field has a value of 2 and the API format is GraphQL.
"api:jsonrpc"	Include this if the interface field has a value of 2 and the API format is JSON-RPC.
"api:mcp"	Include this if the interface field has a value of 2 and the API format is OpenAPI.
"api:a2a"	Include this if the interface field has a value of 2 and the API format is A2A.
"token:erc20"	Include this if the token in the fungibleTokenId field supports ERC-20.
"token:erc3009"	Include this if the token in the fungibleTokenId field supports ERC-3009.
"token:spl"	Include this if the token in the fungibleTokenId field supports SPL.
"token:2022"	Include this if the token in the fungibleTokenId field supports Token-2022.
"token:transferable"	Include this if the token in the fungibleTokenId is transferable.
"token:burnable"	Include this if the token in the fungibleTokenId is burnable.
"pay:x402"	Include this if the endpoint supports x402 payments.
"pay:manual"	Include this if the endpoint supports traditional payments.

Appendix D

Example Test Vectors for Canonicalization+Hashing

The following test vectors were generated using JCS (RFC 8785) canonicalization followed by UTF-8 encoding and hashing. Hashes are 0x-prefixed hex. Both keccak256 and sha256 columns are provided; implementers should verify against the algorithm specified in their dataHashAlgorithm field.

Input JSON	Canonicalized JSON	keccak256 Hash	sha256 Hash
<code>{"a": 1.0, "b": {"c": "\n"}}</code>	<code>{"a":1,"b":{"c":"\n"}}</code>	0x272619e60fdf0b8408352a24263ab5bd43e5c3873828556737960144deb08639	0xa14a36c545cf0d9cd10a13680775cb5b3c5e17d2d426c5a54b8af1d2d17d5351
<code>{"b":2,"a":1}</code>	<code>{"a":1,"b":2}</code>	0xb8ffb64722137f4b100665a52e3c943f8066e8ab8ba3b427e6f4b404defd82b0	0x43258cff783fe7036d8a43033f830adfc60ec037382473548ac742b888292777
<code>{"x":[{"y":true}]}</code>	<code>{"x":[{"y":true}]}</code>	0x2645502e6bc76dd669aa0e22c68b99defe44dd5479159445656a693e57764097	0x01f868b03ac751f2fd0e87fbea94e729866312f476e8fe029c8959e94acd6889

Appendix E

did:handle Specification

The **did:handle** method provides a simple, human-readable DID format for representing platform-specific user or service identifiers (e.g., social platforms, developer platforms, messaging systems).

This method is intended for referential identity only and does not imply authentication, control, or ownership.

E.1 Syntax

A **did:handle** MUST conform to the following format:

did:handle:<platform>:<identifier>

Where:

- **platform** identifies the platform or namespace (e.g., github, twitter, discord)
- **identifier** identifies a user, organization, or resource within that platform

E.2 Canonicalization

To ensure deterministic hashing and comparison, **did:handle** identifiers MUST be canonicalized as follows:

- The DID scheme (**did**) and method name (**handle**) MUST be lowercase.
- The **platform** component MUST be lowercased.
- The **identifier** component MUST preserve its original case and byte representation.
- Additional normalization (including Unicode folding, trimming, or case transformation) MUST NOT be applied to **identifier**.

E.3 Semantics

- **platform** is treated as a namespace identifier and is case-insensitive.
- **identifier** is treated as a platform-defined identifier and MAY be case-sensitive.
- Clients MUST NOT assume that identifiers differing only by case refer to the same entity unless the platform explicitly defines such behavior.

E.4 Usage Notes

- **did:handle** does not define verification or control semantics.
- Trust assertions involving did:handle identifiers SHOULD rely on external attestations, confirmations, or platform-specific verification mechanisms.
- When used with hashing or address-derivation schemes, canonicalization MUST be applied prior to hashing.

E.5 **identifier** Registry

A list of values for **platform** can be found in social-platforms.ts.